

```
In [5]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv('Airbnb NYC 2019.csv')
```

```
In [6]: df = pd.read_csv('Airbnb NYC 2019.csv')
df
```

Out[6]:

	id	name	host_id	host_name	neighbourhood_group	neighbourhood	lati
0	2539	Clean & quiet apt home by the park	2787	John	Brooklyn	Kensington	40.6
1	2595	Skylit Midtown Castle	2845	Jennifer	Manhattan	Midtown	40.7
2	3647	THE VILLAGE OF HARLEM....NEW YORK !	4632	Elisabeth	Manhattan	Harlem	40.8
3	3831	Cozy Entire Floor of Brownstone	4869	LisaRoxanne	Brooklyn	Clinton Hill	40.6
4	5022	Entire Apt: Spacious Studio/Loft by central park	7192	Laura	Manhattan	East Harlem	40.7
...	...	...	...	...	...	...	...
48890	36484665	Charming one bedroom - newly renovated rowhouse	8232441	Sabrina	Brooklyn	Bedford-Stuyvesant	40.6
48891	36485057	Affordable room in Bushwick/East Williamsburg	6570630	Marisol	Brooklyn	Bushwick	40.7
48892	36485431	Sunny Studio at Historical Neighborhood	23492952	Ilgar & Aysel	Manhattan	Harlem	40.8
48893	36485609	43rd St. Time Square-cozy single bed	30985759	Taz	Manhattan	Hell's Kitchen	40.7
48894	36487245	Trendy duplex in the very heart of Hell's Kitchen	68119814	Christophe	Manhattan	Hell's Kitchen	40.7

48895 rows × 15 columns

```
In [7]: df.head()
```

Out[7]:

	id	name	host_id	host_name	neighbourhood_group	neighbourhood	latitude	long
0	2539	Clean & quiet apt home by the park	2787	John	Brooklyn	Kensington	40.64749	-73.9
1	2595	Skylit Midtown Castle	2845	Jennifer	Manhattan	Midtown	40.75362	-73.9
2	3647	THE VILLAGE OF HARLEM....NEW YORK !	4632	Elisabeth	Manhattan	Harlem	40.80902	-73.9
3	3831	Cozy Entire Floor of Brownstone	4869	LisaRoxanne	Brooklyn	Clinton Hill	40.68514	-73.9
4	5022	Entire Apt: Spacious Studio/Loft by central park	7192	Laura	Manhattan	East Harlem	40.79851	-73.9

In [8]:

df.columns

Out[8]:

Index(['id', 'name', 'host_id', 'host_name', 'neighbourhood_group', 'neighbourhood', 'latitude', 'longitude', 'room_type', 'price', 'minimum_nights', 'number_of_reviews', 'reviews_per_month', 'calculated_host_listings_count', 'availability_365'], dtype='object')

In [9]:

df.isnull()

Out[9]:

	id	name	host_id	host_name	neighbourhood_group	neighbourhood	latitude	longitude
0	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...
48890	False	False	False	False	False	False	False	False
48891	False	False	False	False	False	False	False	False
48892	False	False	False	False	False	False	False	False
48893	False	False	False	False	False	False	False	False
48894	False	False	False	False	False	False	False	False

48895 rows × 15 columns

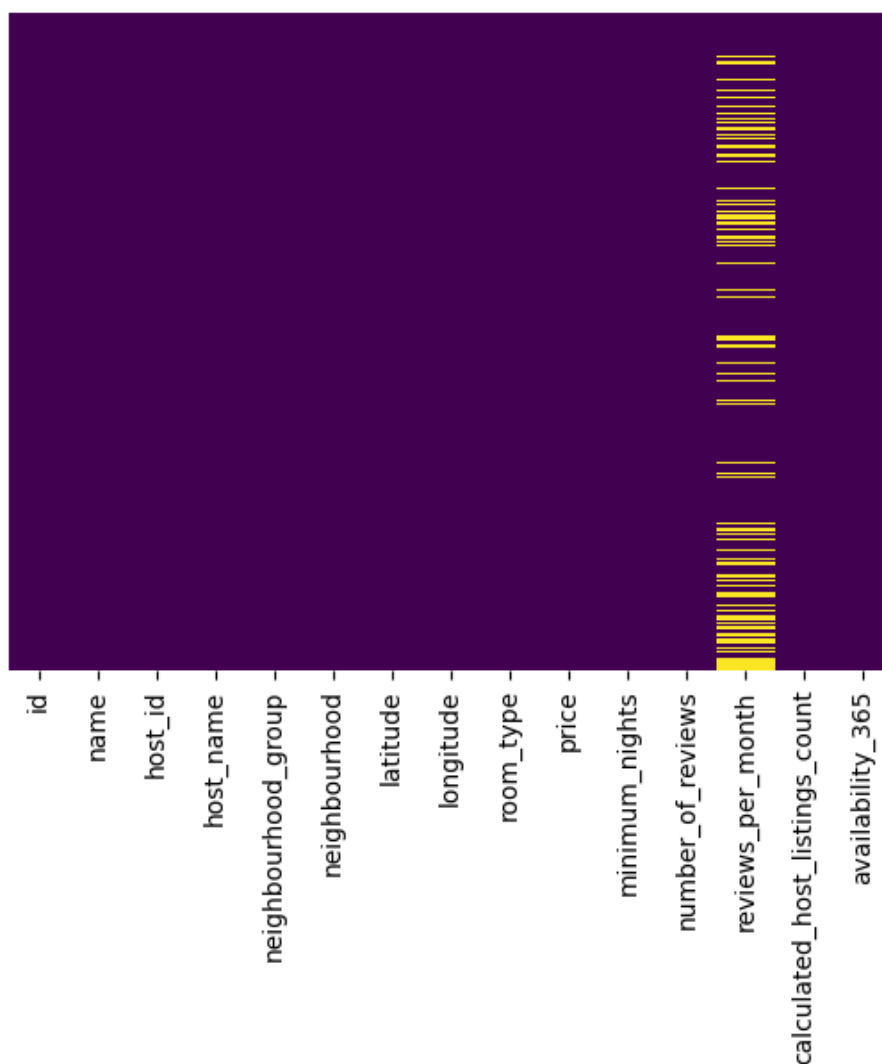
In [10]:

df.isnull().sum()

```
Out[10]: id                0
         name              16
         host_id           0
         host_name         21
         neighbourhood_group 0
         neighbourhood      0
         latitude           0
         longitude          0
         room_type          0
         price              0
         minimum_nights     0
         number_of_reviews  0
         reviews_per_month 10052
         calculated_host_listings_count 0
         availability_365    0
         dtype: int64
```

```
In [11]: sns.heatmap(df.isnull(), cbar=False, cmap='viridis', yticklabels=False, xticklabels=True)
```

```
Out[11]: <Axes: >
```



```
In [12]: df = df.dropna().reset_index(drop=True)
```

```
In [13]: df.shape
```

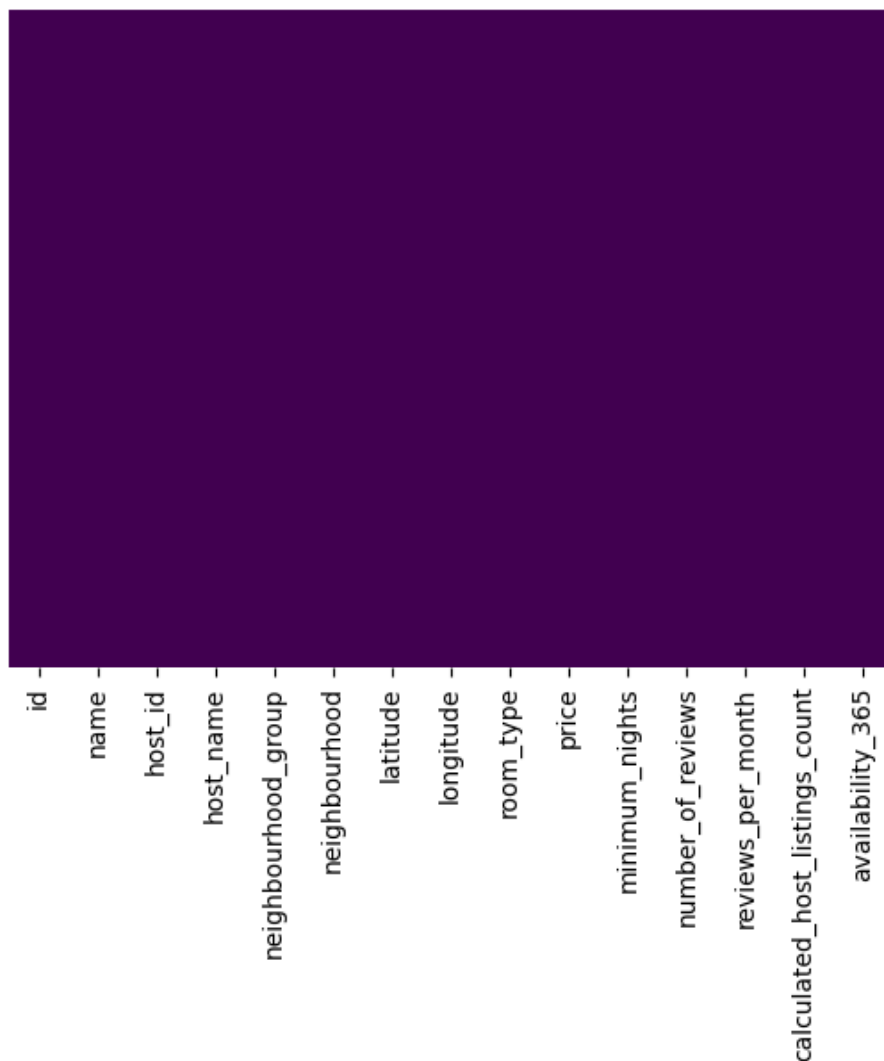
```
Out[13]: (38821, 15)
```

```
In [14]: df.isnull().sum()
```

```
Out[14]: id          0
         name        0
         host_id     0
         host_name    0
         neighbourhood_group  0
         neighbourhood  0
         latitude     0
         longitude    0
         room_type    0
         price        0
         minimum_nights  0
         number_of_reviews  0
         reviews_per_month  0
         calculated_host_listings_count  0
         availability_365  0
         dtype: int64
```

```
In [15]: sns.heatmap(df.isnull(), cbar=False, cmap='viridis', yticklabels=False, xticklabels=True)
```

```
Out[15]: <Axes: >
```



```
In [16]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 38821 entries, 0 to 38820
Data columns (total 15 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   id                                    38821 non-null  int64
1   name                                38821 non-null  object
2   host_id                             38821 non-null  int64
3   host_name                           38821 non-null  object
4   neighbourhood_group                 38821 non-null  object
5   neighbourhood                       38821 non-null  object
6   latitude                           38821 non-null  float64
7   longitude                           38821 non-null  float64
8   room_type                           38821 non-null  object
9   price                               38821 non-null  int64
10  minimum_nights                      38821 non-null  int64
11  number_of_reviews                   38821 non-null  int64
12  reviews_per_month                  38821 non-null  float64
13  calculated_host_listings_count      38821 non-null  int64
14  availability_365                    38821 non-null  int64
dtypes: float64(3), int64(7), object(5)
memory usage: 4.4+ MB
```

In [17]: `df.describe()`

Out[17]:

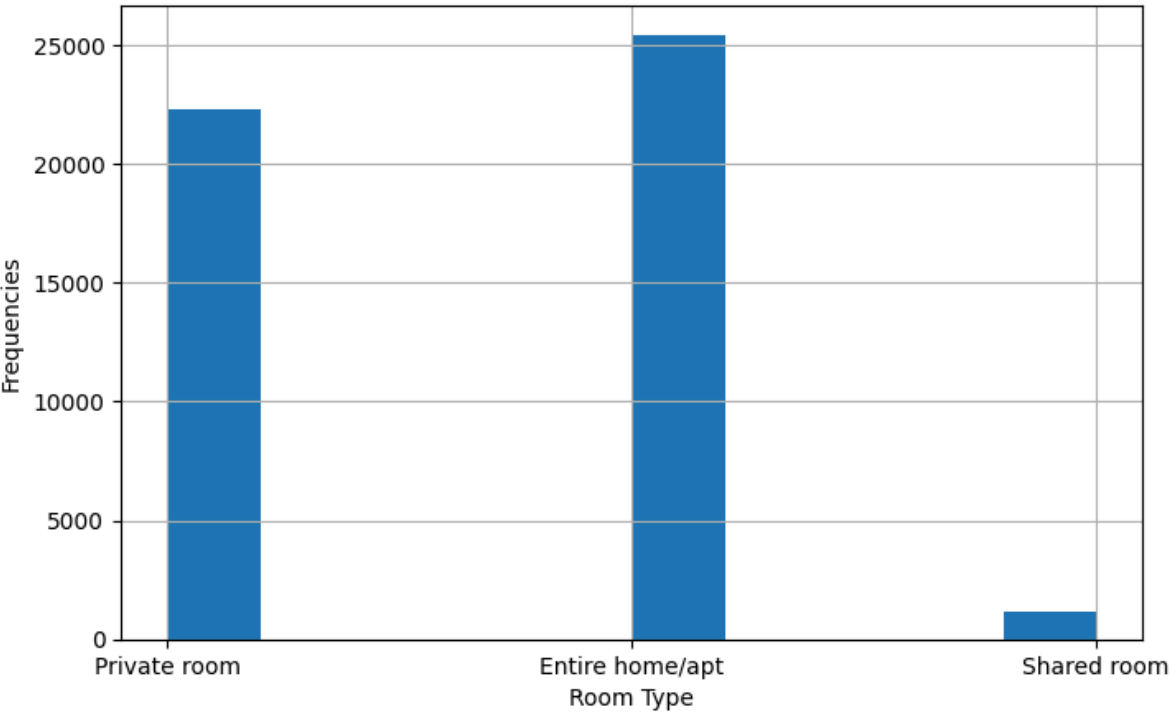
	id	host_id	latitude	longitude	price	minimum_nights	nurr
count	3.882100e+04	3.882100e+04	38821.000000	38821.000000	38821.000000	38821.000000	
mean	1.810081e+07	6.424582e+07	40.728129	-73.951149	142.332526	5.869220	
std	1.069372e+07	7.589752e+07	0.054991	0.046693	196.994756	17.389026	
min	2.539000e+03	2.438000e+03	40.506410	-74.244420	0.000000	1.000000	
25%	8.721444e+06	7.029525e+06	40.688640	-73.982460	69.000000	1.000000	
50%	1.887286e+07	2.837092e+07	40.721710	-73.954810	101.000000	2.000000	
75%	2.756746e+07	1.018905e+08	40.762990	-73.935020	170.000000	4.000000	
max	3.645581e+07	2.738417e+08	40.913060	-73.712990	10000.000000	1250.000000	

In [18]: `import pandas as pd`
`df = pd.read_csv('Airbnb NYC 2019.csv')`
`df['room_type'].value_counts()`

Out[18]:

```
room_type
Entire home/apt    25409
Private room       22326
Shared room        1160
Name: count, dtype: int64
```

In [19]: `import matplotlib.pyplot as plt`
`plt.figure(figsize=(8,5))`
`plt.xticks(rotation=0)`
`df['room_type'].hist()`
`plt.xlabel('Room Type')`
`plt.ylabel('Frequencies')`
`plt.show()`



```
In [20]: def price_category(price):
          if price < 50:
              return 'Low'
          elif price <= 150:
              return 'Medium'
          else:
              return 'High'

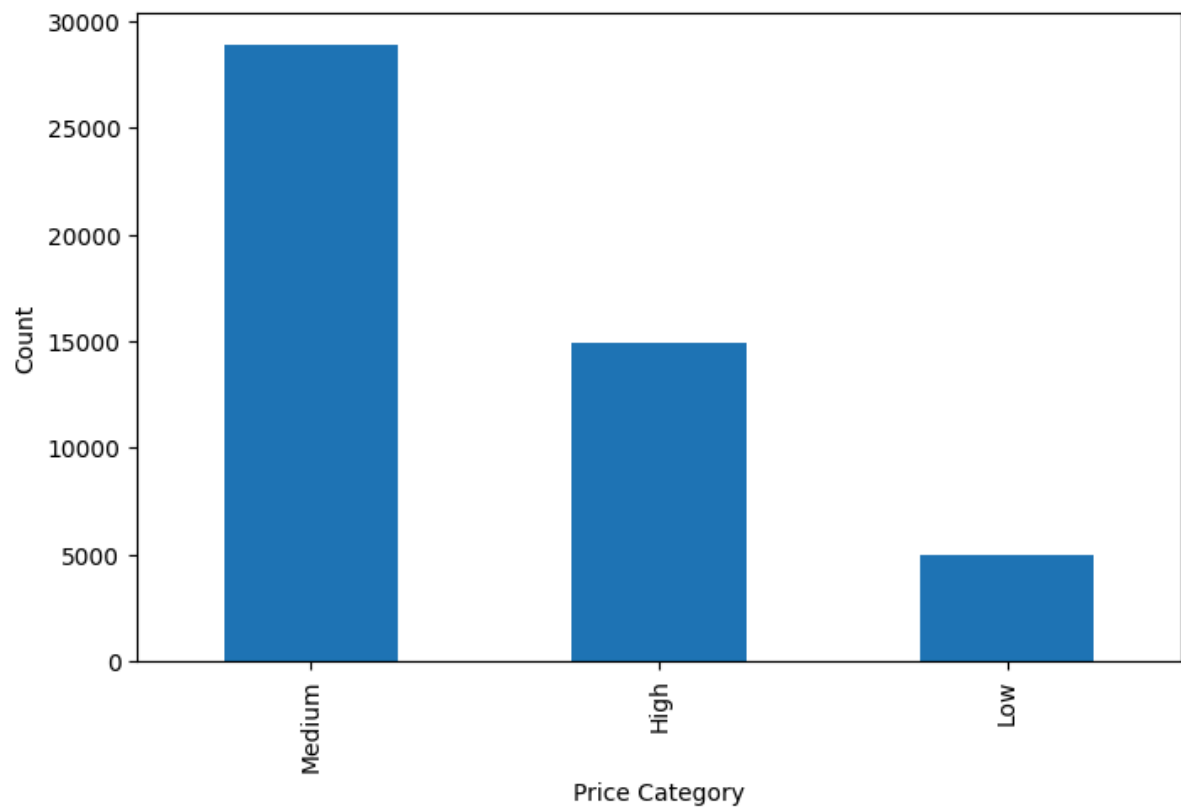
          df['price_category'] = df['price'].apply(price_category)
          df[['price', 'price_category']]
```

Out[20]:

	price	price_category
0	149	Medium
1	225	High
2	150	Medium
3	89	Medium
4	80	Medium
...	...	...
48890	70	Medium
48891	40	Low
48892	115	Medium
48893	55	Medium
48894	90	Medium

48895 rows × 2 columns

```
In [21]: import matplotlib.pyplot as plt
          plt.figure(figsize=(8,5))
          df['price_category'].value_counts().plot(kind='bar')
          plt.xlabel('Price Category')
          plt.ylabel('Count')
          plt.show()
```

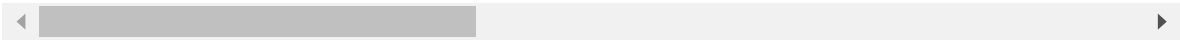


```
In [22]: df.to_csv('modified_data.csv', index=False)
df
```

Out[22]:

	id	name	host_id	host_name	neighbourhood_group	neighbourhood	lati
0	2539	Clean & quiet apt home by the park	2787	John	Brooklyn	Kensington	40.6
1	2595	Skylit Midtown Castle	2845	Jennifer	Manhattan	Midtown	40.7
2	3647	THE VILLAGE OF HARLEM....NEW YORK !	4632	Elisabeth	Manhattan	Harlem	40.8
3	3831	Cozy Entire Floor of Brownstone	4869	LisaRoxanne	Brooklyn	Clinton Hill	40.6
4	5022	Entire Apt: Spacious Studio/Loft by central park	7192	Laura	Manhattan	East Harlem	40.7
...	...	...	...	...	...	...	...
48890	36484665	Charming one bedroom - newly renovated rowhouse	8232441	Sabrina	Brooklyn	Bedford-Stuyvesant	40.6
48891	36485057	Affordable room in Bushwick/East Williamsburg	6570630	Marisol	Brooklyn	Bushwick	40.7
48892	36485431	Sunny Studio at Historical Neighborhood	23492952	Ilgar & Aysel	Manhattan	Harlem	40.8
48893	36485609	43rd St. Time Square-cozy single bed	30985759	Taz	Manhattan	Hell's Kitchen	40.7
48894	36487245	Trendy duplex in the very heart of Hell's Kitchen	68119814	Christophe	Manhattan	Hell's Kitchen	40.7

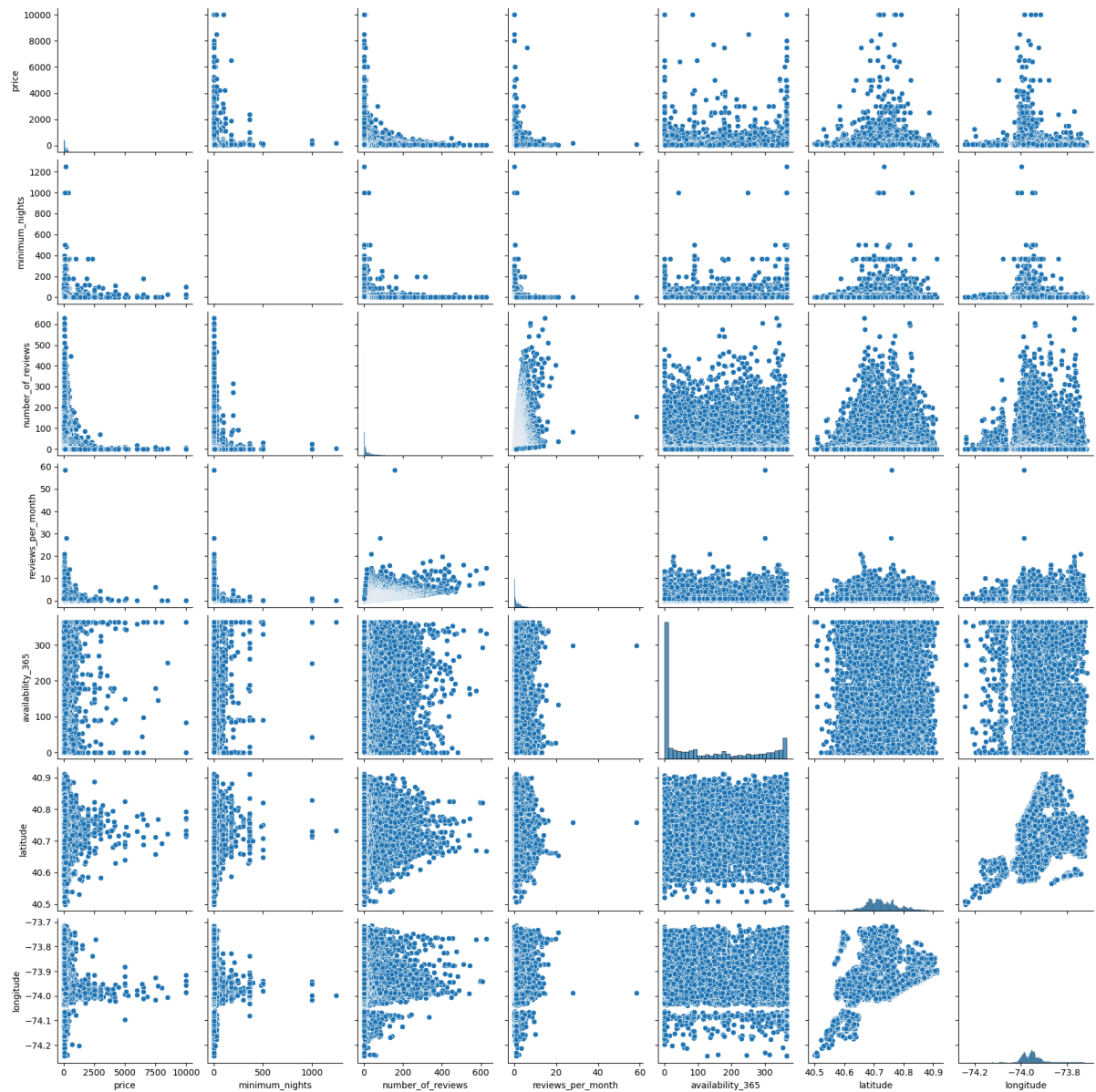
48895 rows × 16 columns



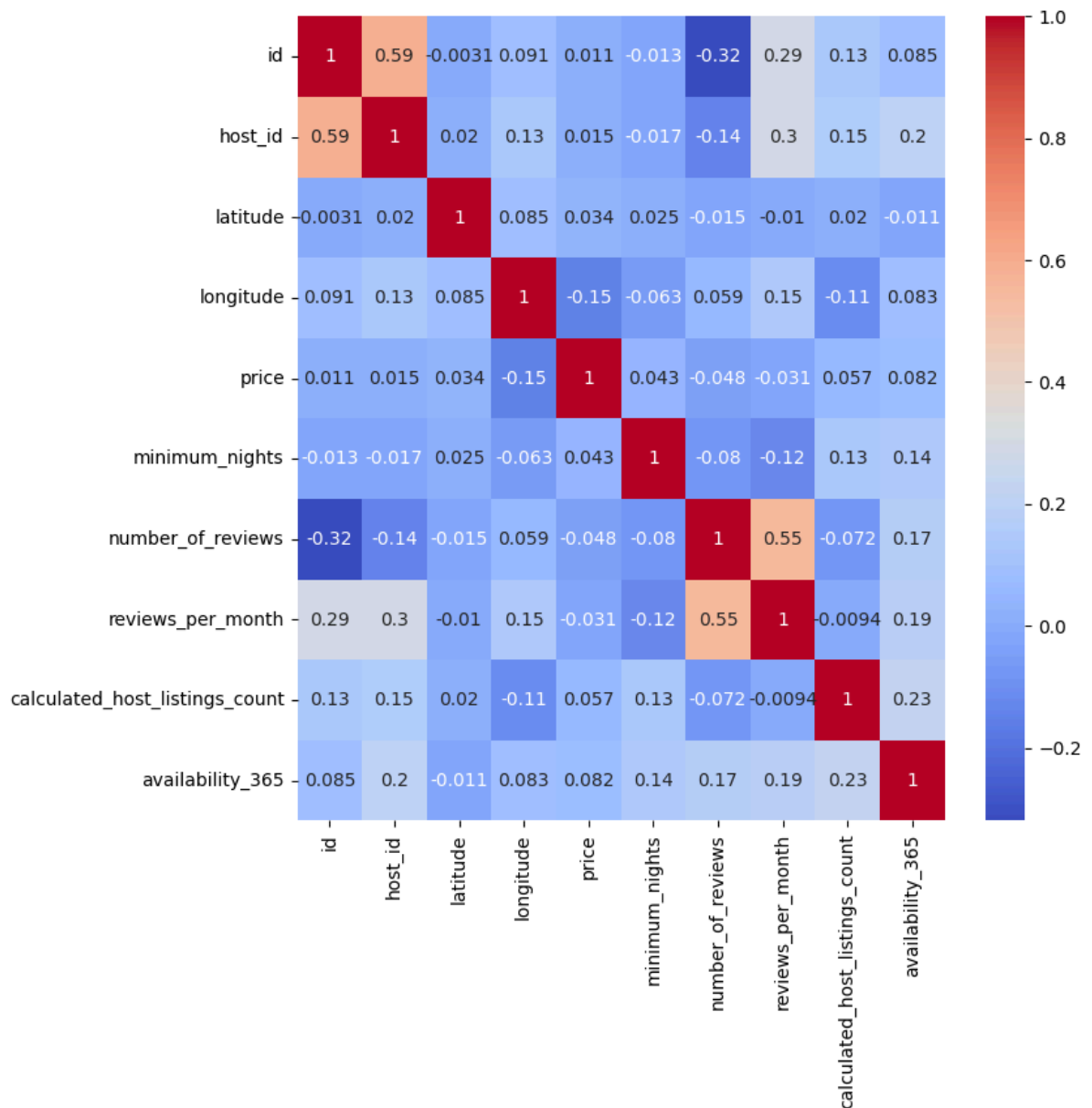
In [23]:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv('Airbnb NYC 2019.csv')
num_cols = ['price', 'minimum_nights', 'number_of_reviews', 'reviews_per_month', 'availability_365']
sns.pairplot(df[num_cols])
plt.show()
```

```
In [24]: import pandas as pd, seaborn as sns, matplotlib.pyplot as plt
df = pd.read_csv('Airbnb NYC 2019.csv')
num_df = df.select_dtypes(include=['float64', 'int64'])
corr = num_df.corr()
plt.figure(figsize=(8,8))
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.show()
```



```
In [25]: from sklearn.model_selection import train_test_split

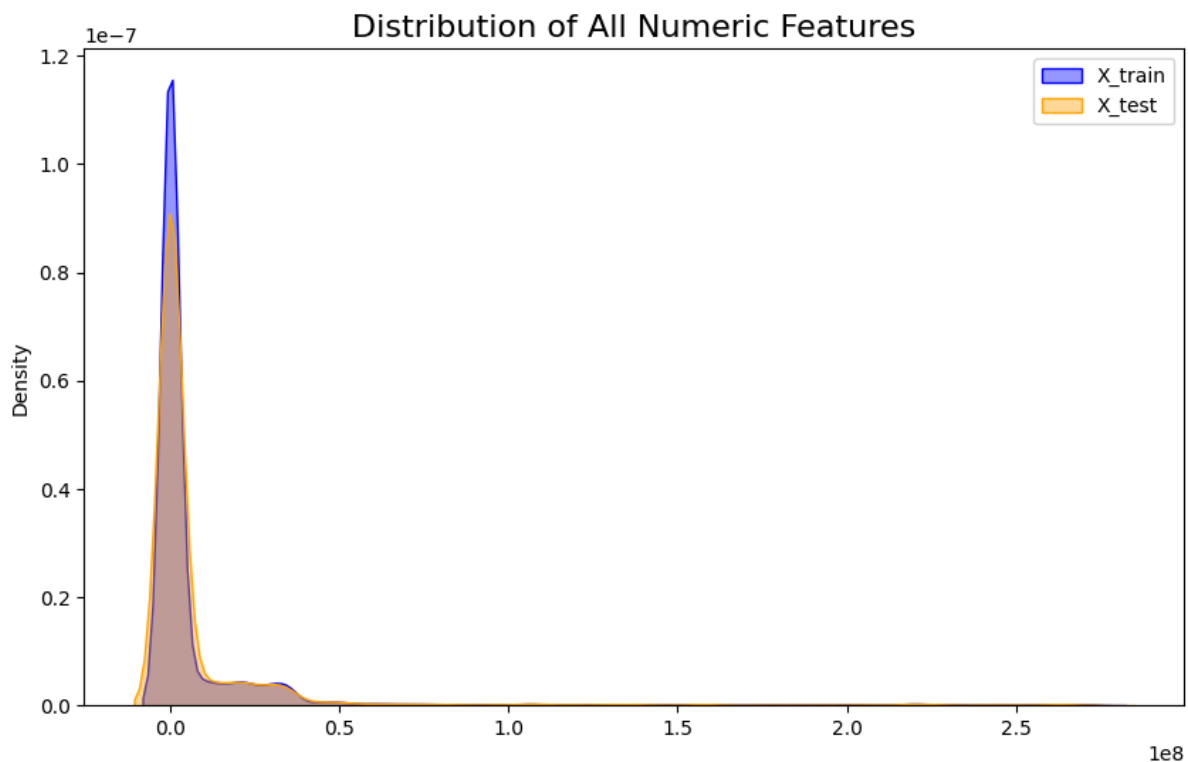
X = df.drop('price', axis=1) # Features (drop target column)
Y = df['price']              # Target
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=70)
print(X_train.shape, X_test.shape, Y_train.shape, Y_test.shape)
```

```
(39116, 14) (9779, 14) (39116,) (9779,)
```

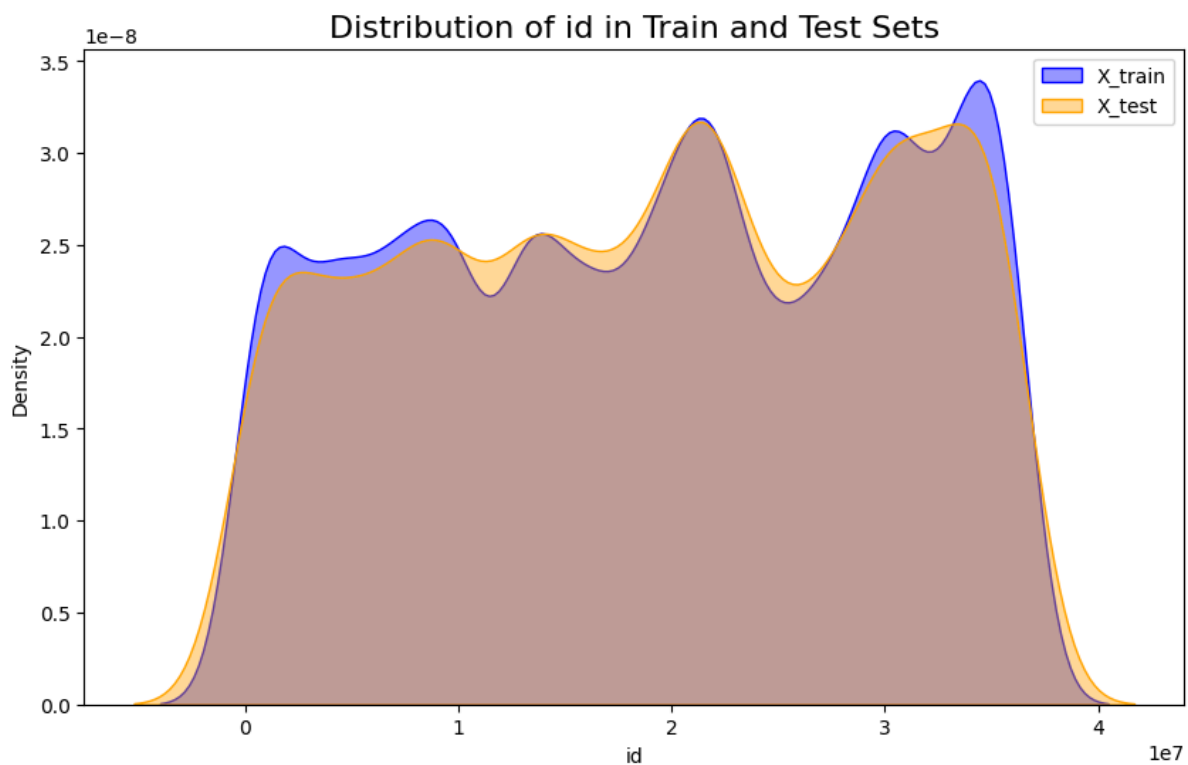
```
In [26]: import numpy as np

# Select only numeric columns
X_train_num = X_train.select_dtypes(include=[np.number])
X_test_num = X_test.select_dtypes(include=[np.number])

plt.figure(figsize=(10,6))
sns.kdeplot(X_train_num.values.flatten(), color="blue", label="X_train", fill=True, alpha=0.5)
sns.kdeplot(X_test_num.values.flatten(), color="orange", label="X_test", fill=True, alpha=0.5)
plt.title("Distribution of All Numeric Features", fontsize=16)
plt.legend()
plt.show()
```



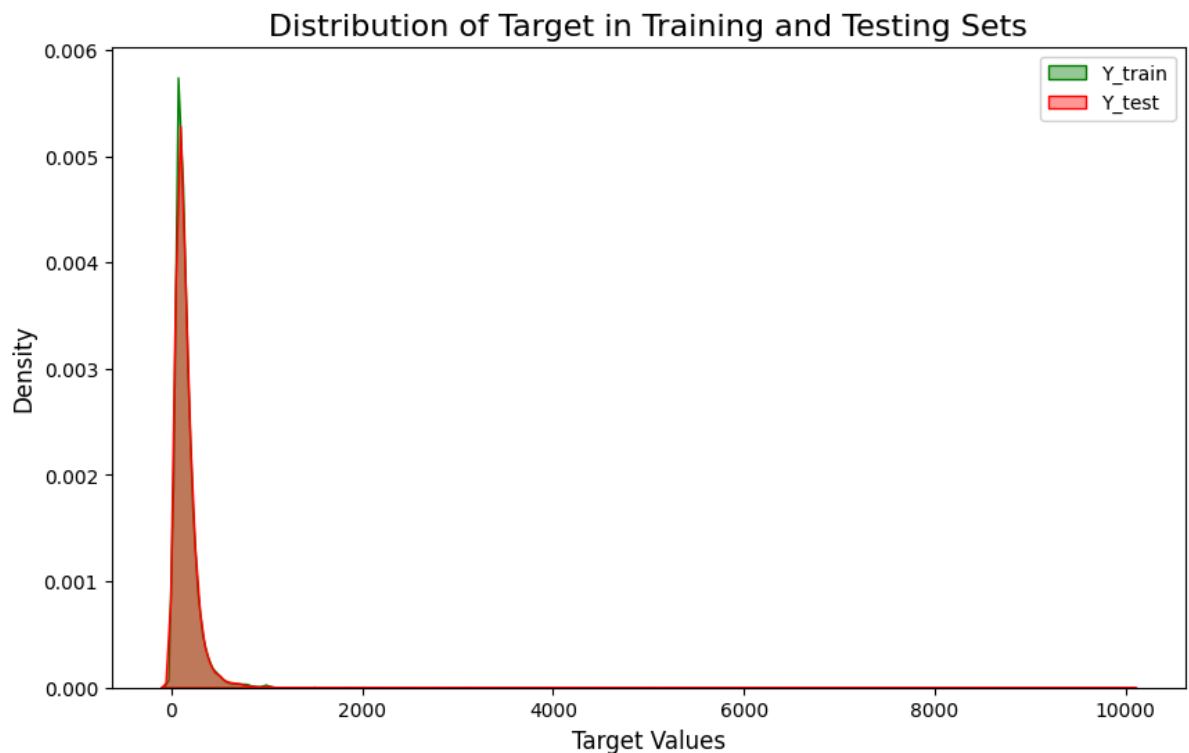
```
In [27]: feature = X_train.columns[0] # pick first column or any you want
plt.figure(figsize=(10,6))
sns.kdeplot(X_train[feature], color="blue", label="X_train", fill=True, alpha=0.4)
sns.kdeplot(X_test[feature], color="orange", label="X_test", fill=True, alpha=0.4)
plt.title(f"Distribution of {feature} in Train and Test Sets", fontsize=16)
plt.legend()
plt.show()
```



```
In [28]: plt.figure(figsize=(10,6))
sns.kdeplot(Y_train, color="green", label="Y_train", fill=True, alpha=0.4)
sns.kdeplot(Y_test, color="red", label="Y_test", fill=True, alpha=0.4)

plt.title("Distribution of Target in Training and Testing Sets", fontsize=16)
plt.xlabel("Target Values", fontsize=12)
```

```
plt.ylabel("Density", fontsize=12)
plt.legend()
plt.show()
```



```
In [29]: import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.svm import SVR
from sklearn.neighbors import KNeighborsRegressor

# Load dataset
df = pd.read_csv("Airbnb NYC 2019.csv")

# Select features and target
# Drop non-numeric and irrelevant columns
X = df.drop(columns=['price', 'name', 'host_name', 'reviews_per_month'])
X = pd.get_dummies(X, drop_first=True) # Encode categorical columns
y = df['price']

# Remove extreme outliers for price
y = np.clip(y, 0, 1000)

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Initialize models
regressors = {
    "Linear Regression": LinearRegression(),
    "Ridge Regression": Ridge(),
    "Lasso Regression": Lasso(),
    "Decision Tree": DecisionTreeRegressor(),
```

```
"Random Forest": RandomForestRegressor(),
"Gradient Boosting": GradientBoostingRegressor(),
"Support Vector Machine": SVR(),
"K-Nearest Neighbors": KNeighborsRegressor()
}

# Train and evaluate
for name, model in regressors.items():
    print(f"\nTraining {name}...")
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)

    mae = mean_absolute_error(y_test, y_pred)
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, y_pred)

    print(f"{name} MAE: {mae:.2f}")
    print(f"{name} MSE: {mse:.2f}")
    print(f"{name} RMSE: {rmse:.2f}")
    print(f"{name} R²: {r2:.2f}")
```

```
Training Linear Regression...
Linear Regression MAE: 58.23
Linear Regression MSE: 10918.48
Linear Regression RMSE: 104.49
Linear Regression R²: 0.33
```

```
Training Ridge Regression...
Ridge Regression MAE: 58.23
Ridge Regression MSE: 10918.49
Ridge Regression RMSE: 104.49
Ridge Regression R²: 0.33
```

```
Training Lasso Regression...
Lasso Regression MAE: 57.88
Lasso Regression MSE: 10974.02
Lasso Regression RMSE: 104.76
Lasso Regression R²: 0.33
```

```
Training Decision Tree...
Decision Tree MAE: 68.15
Decision Tree MSE: 17883.12
Decision Tree RMSE: 133.73
Decision Tree R²: -0.10
```

```
Training Random Forest...
Random Forest MAE: 51.22
Random Forest MSE: 8967.83
Random Forest RMSE: 94.70
Random Forest R²: 0.45
```

```
Training Gradient Boosting...
Gradient Boosting MAE: 53.64
Gradient Boosting MSE: 9722.25
Gradient Boosting RMSE: 98.60
Gradient Boosting R²: 0.40
```

```
Training Support Vector Machine...
Support Vector Machine MAE: 54.45
Support Vector Machine MSE: 12936.80
Support Vector Machine RMSE: 113.74
Support Vector Machine R²: 0.21
```

```
Training K-Nearest Neighbors...
K-Nearest Neighbors MAE: 57.33
K-Nearest Neighbors MSE: 11210.69
K-Nearest Neighbors RMSE: 105.88
K-Nearest Neighbors R²: 0.31
```

```
In [30]: df = df.select_dtypes(exclude=['object'])
```

```
In [31]: df = pd.get_dummies(df, drop_first=True)
```

```
In [32]: from sklearn.model_selection import train_test_split

X = df.drop('price', axis=1) # replace with your actual target column name
y = df['price']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
In [ ]: from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.svm import SVR
from sklearn.neighbors import KNeighborsRegressor

regressors = {
    "Linear Regression": LinearRegression(),
```

```

    "Ridge Regression": Ridge(),
    "Lasso Regression": Lasso(),
    "Decision Tree": DecisionTreeRegressor(),
    "Random Forest": RandomForestRegressor(),
    "Gradient Boosting": GradientBoostingRegressor(),
    "Support Vector Machine": SVR(),
    "K-Nearest Neighbors": KNeighborsRegressor()
}

```

```

In [40]: import seaborn as sns
import matplotlib.pyplot as plt
for name, model in regressors.items():
    print(f"\nTraining {name}...")
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)

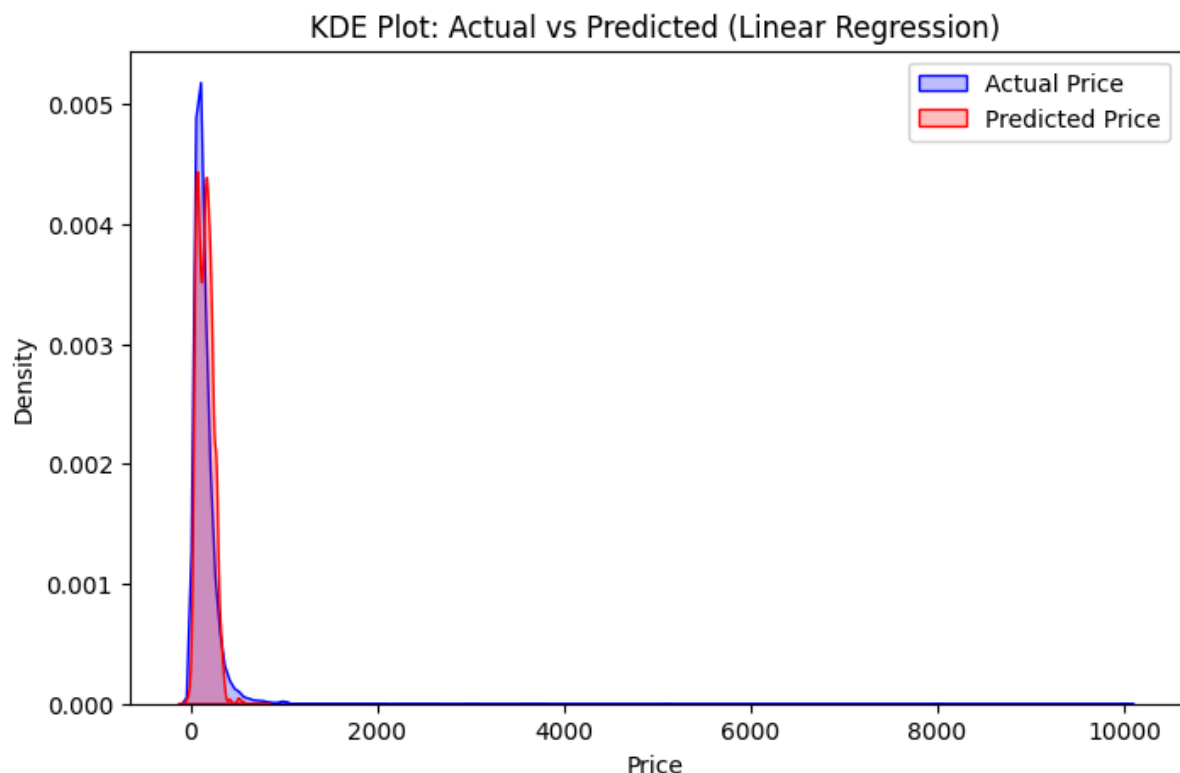
    mae = mean_absolute_error(y_test, y_pred)
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, y_pred)

    print(f"{name} MAE: {mae:.2f}")
    print(f"{name} MSE: {mse:.2f}")
    print(f"{name} RMSE: {rmse:.2f}")
    print(f"{name} R²: {r2:.2f}")

# KDE plot for actual vs predicted prices
plt.figure(figsize=(8,5))
sns.kdeplot(y_test, label="Actual Price", fill=True, color="blue")
sns.kdeplot(y_pred, label="Predicted Price", fill=True, color="red")
plt.title(f"KDE Plot: Actual vs Predicted ({name})")
plt.xlabel("Price")
plt.ylabel("Density")
plt.legend()
plt.show()

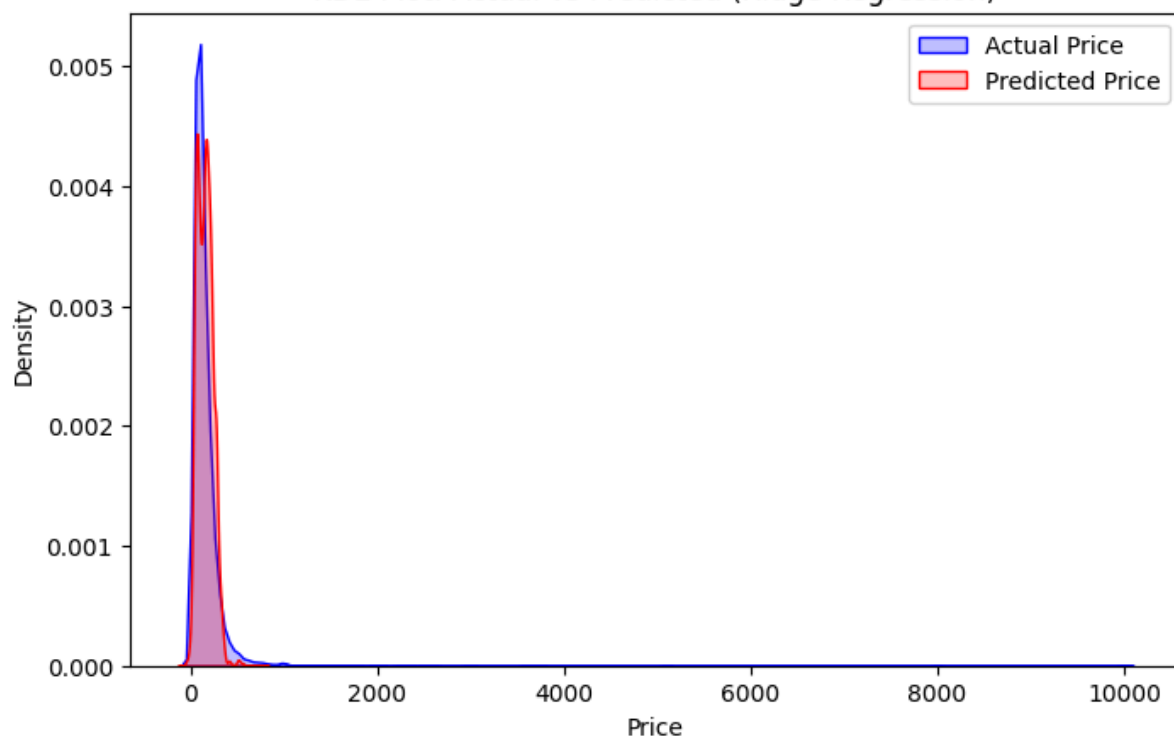
```

Training Linear Regression...
 Linear Regression MAE: 68.01
 Linear Regression MSE: 38219.43
 Linear Regression RMSE: 195.50
 Linear Regression R²: 0.14



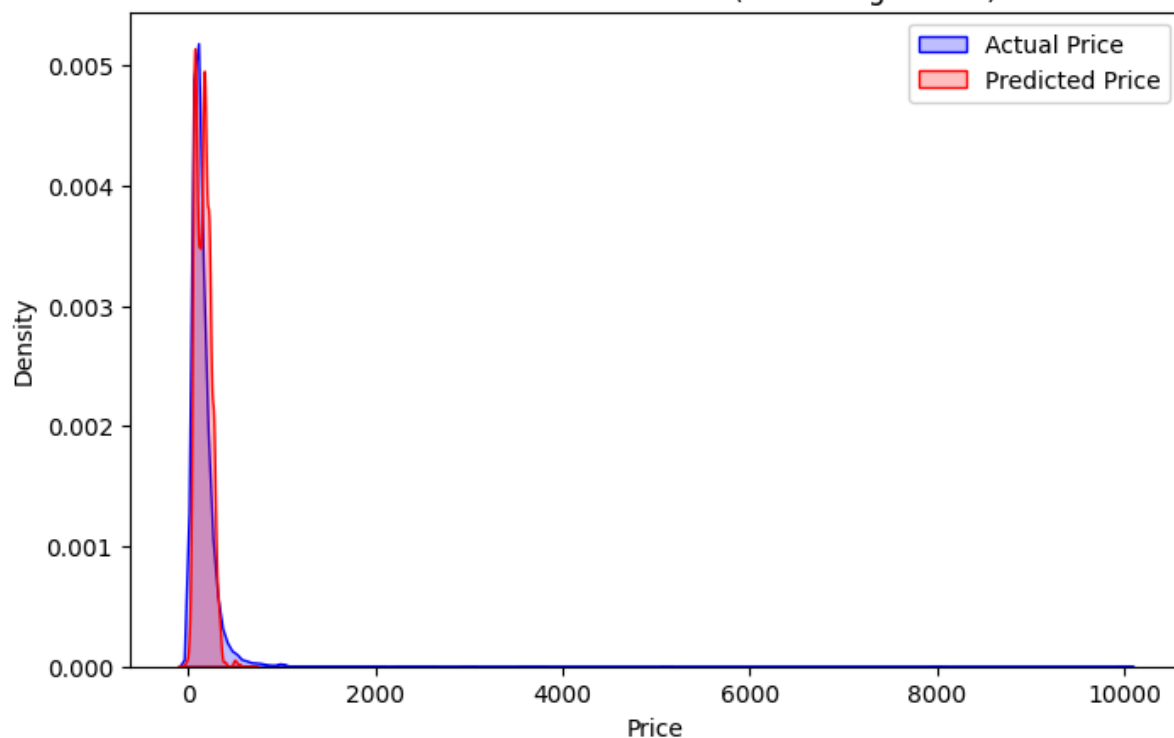
Training Ridge Regression...
Ridge Regression MAE: 68.01
Ridge Regression MSE: 38219.46
Ridge Regression RMSE: 195.50
Ridge Regression R^2 : 0.14

KDE Plot: Actual vs Predicted (Ridge Regression)

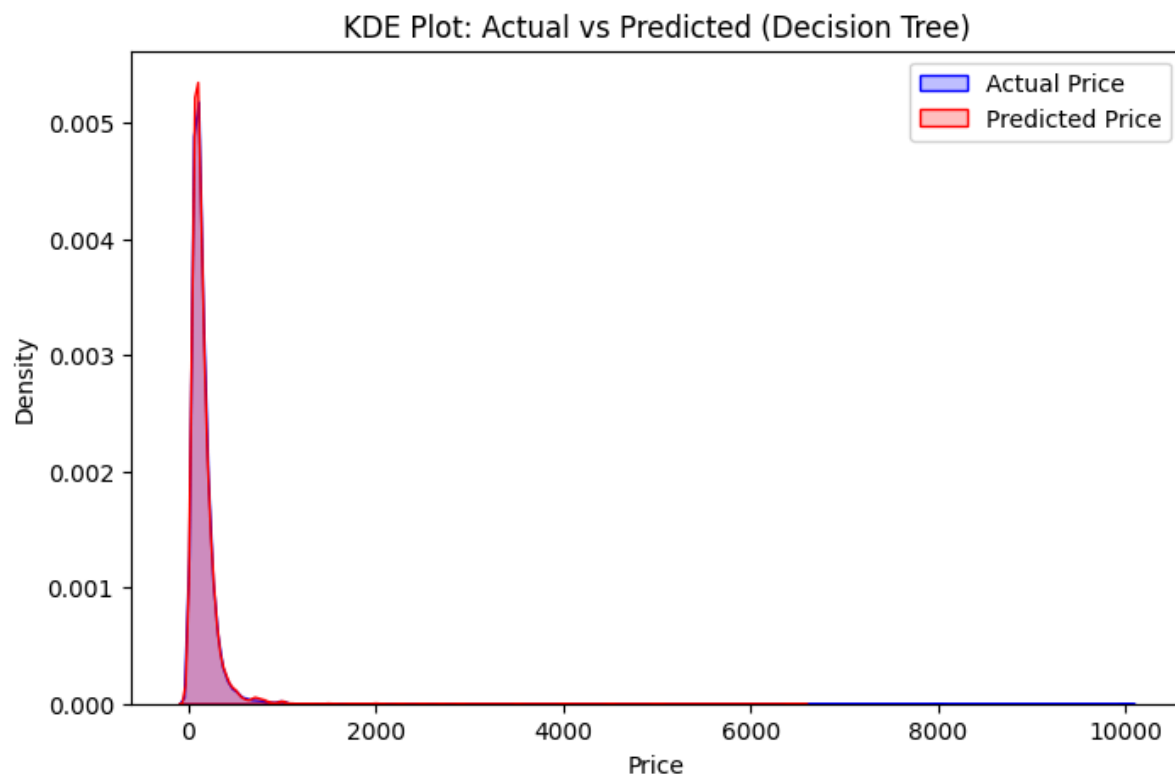


Training Lasso Regression...
Lasso Regression MAE: 67.24
Lasso Regression MSE: 38221.50
Lasso Regression RMSE: 195.50
Lasso Regression R^2 : 0.14

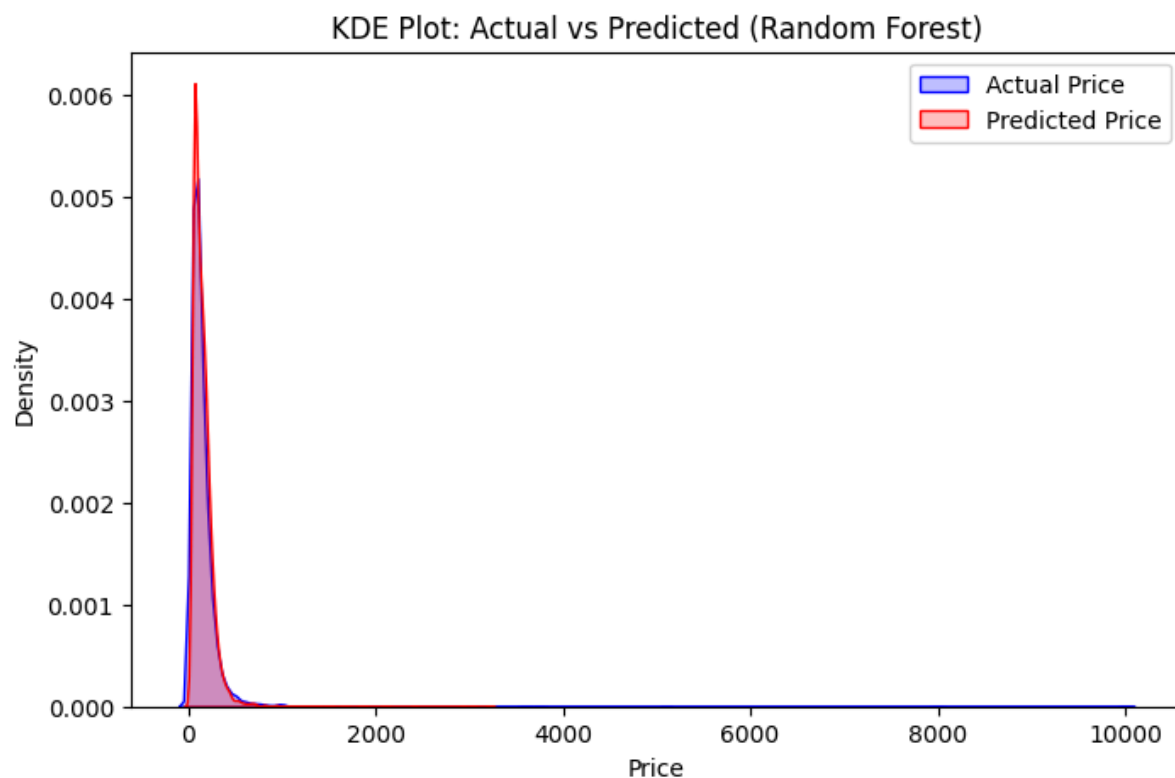
KDE Plot: Actual vs Predicted (Lasso Regression)



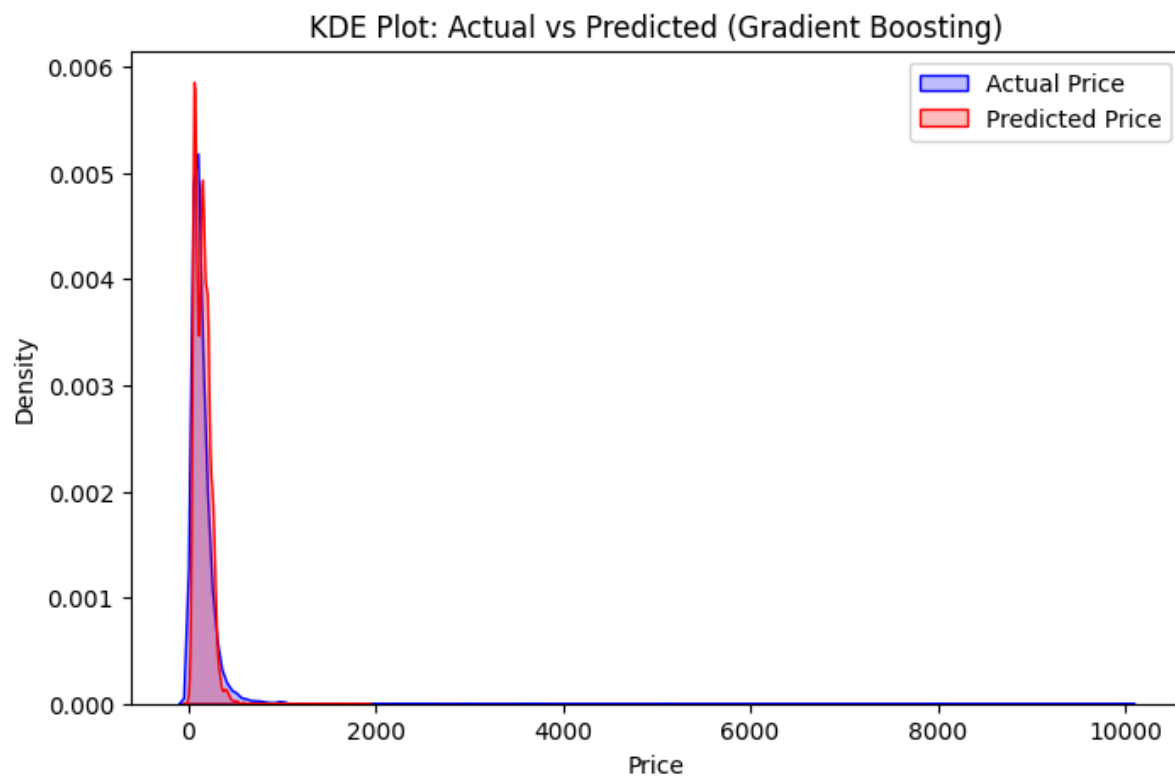
Training Decision Tree...
Decision Tree MAE: 79.79
Decision Tree MSE: 65194.89
Decision Tree RMSE: 255.33
Decision Tree R^2 : -0.47



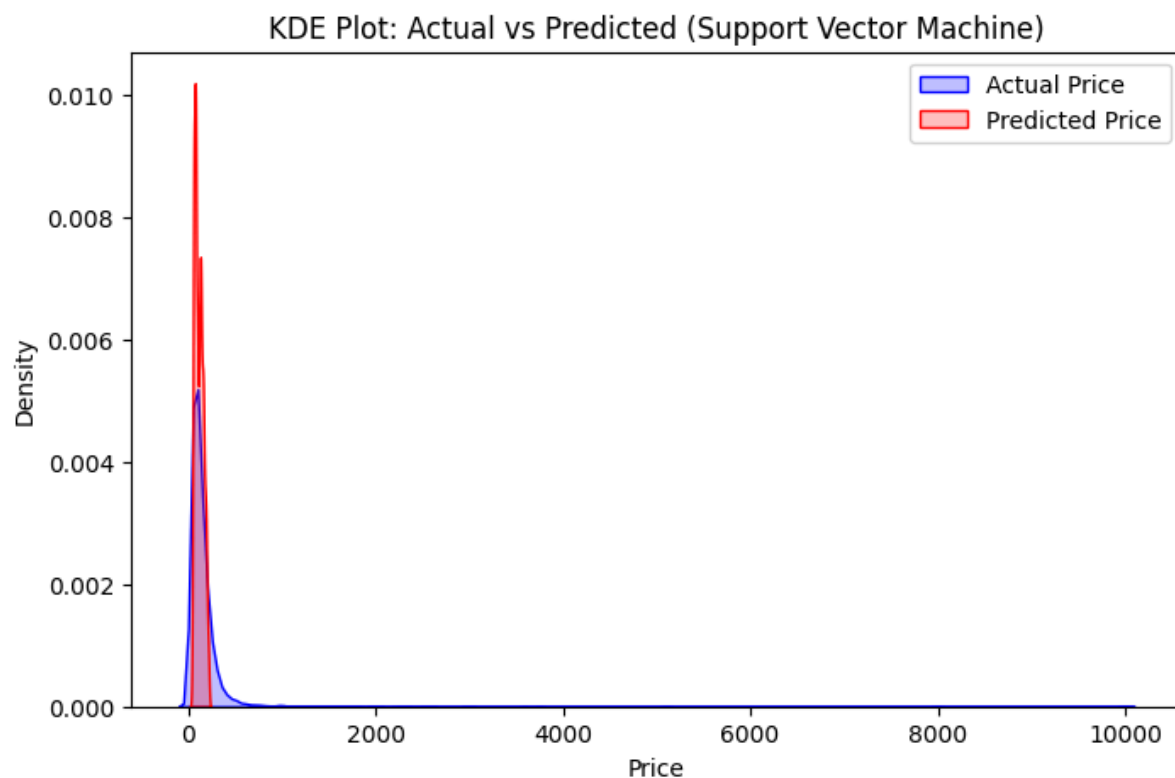
Training Random Forest...
Random Forest MAE: 60.97
Random Forest MSE: 36043.33
Random Forest RMSE: 189.85
Random Forest R^2 : 0.19



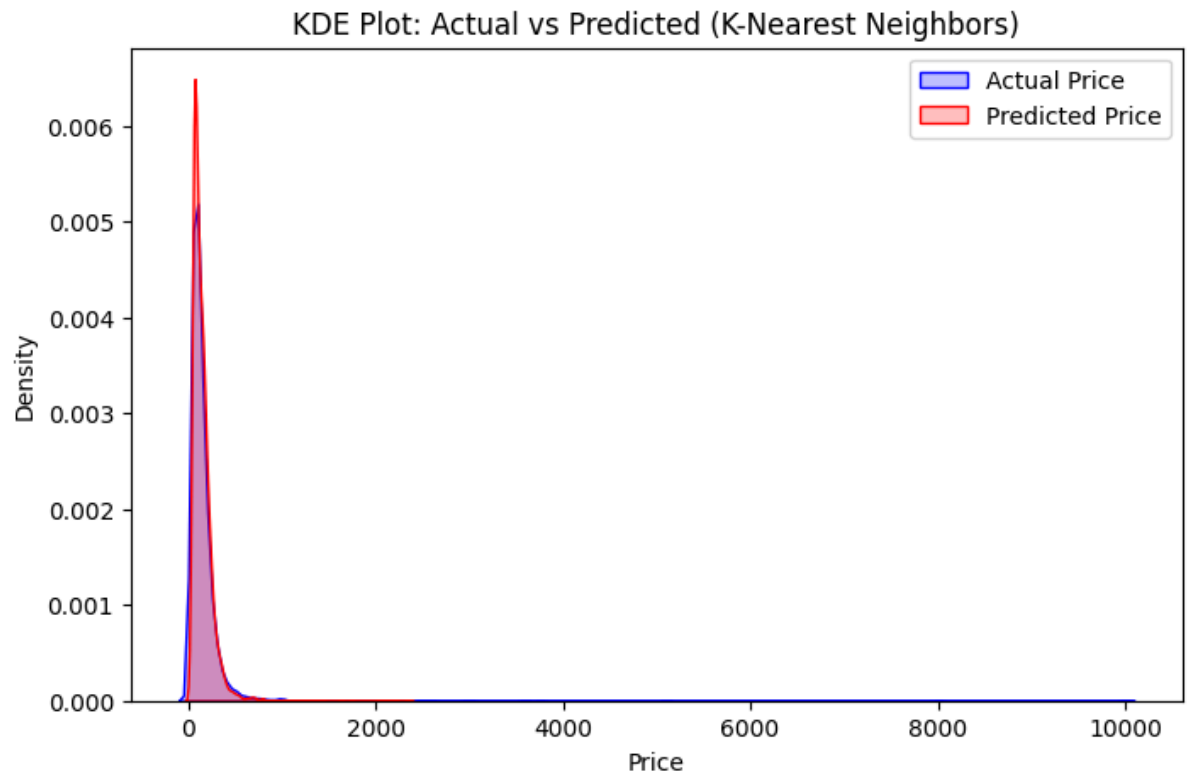
Training Gradient Boosting...
Gradient Boosting MAE: 62.97
Gradient Boosting MSE: 36799.70
Gradient Boosting RMSE: 191.83
Gradient Boosting R^2 : 0.17



Training Support Vector Machine...
Support Vector Machine MAE: 59.54
Support Vector Machine MSE: 40767.50
Support Vector Machine RMSE: 201.91
Support Vector Machine R^2 : 0.08



```
Training K-Nearest Neighbors...  
K-Nearest Neighbors MAE: 66.88  
K-Nearest Neighbors MSE: 42093.95  
K-Nearest Neighbors RMSE: 205.17  
K-Nearest Neighbors R2: 0.05
```

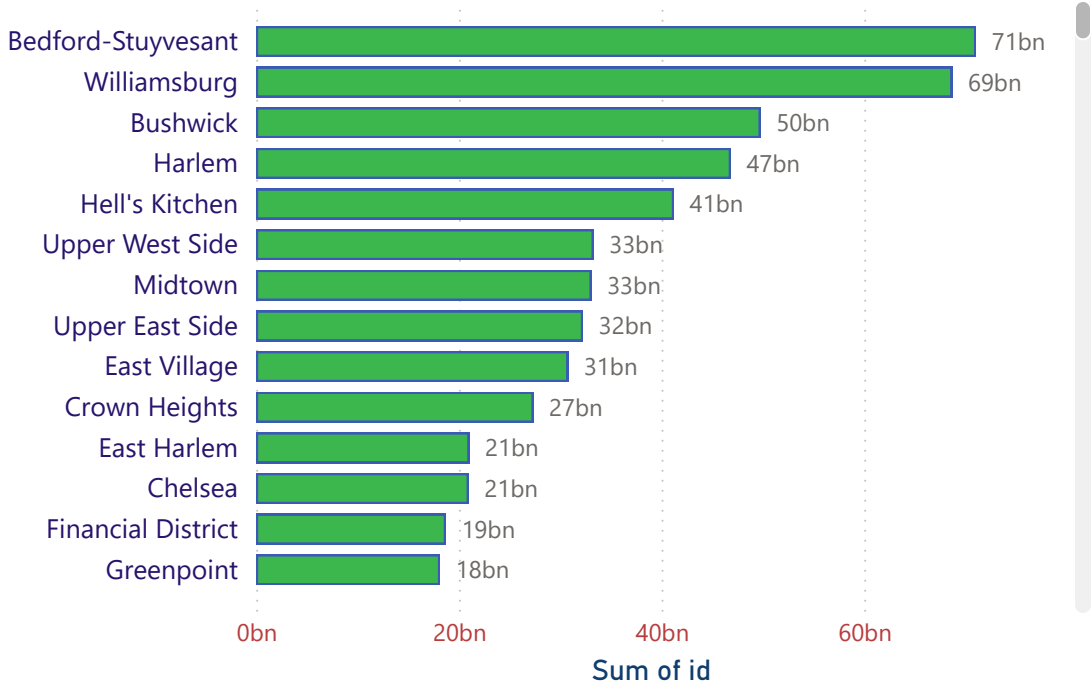


In []:

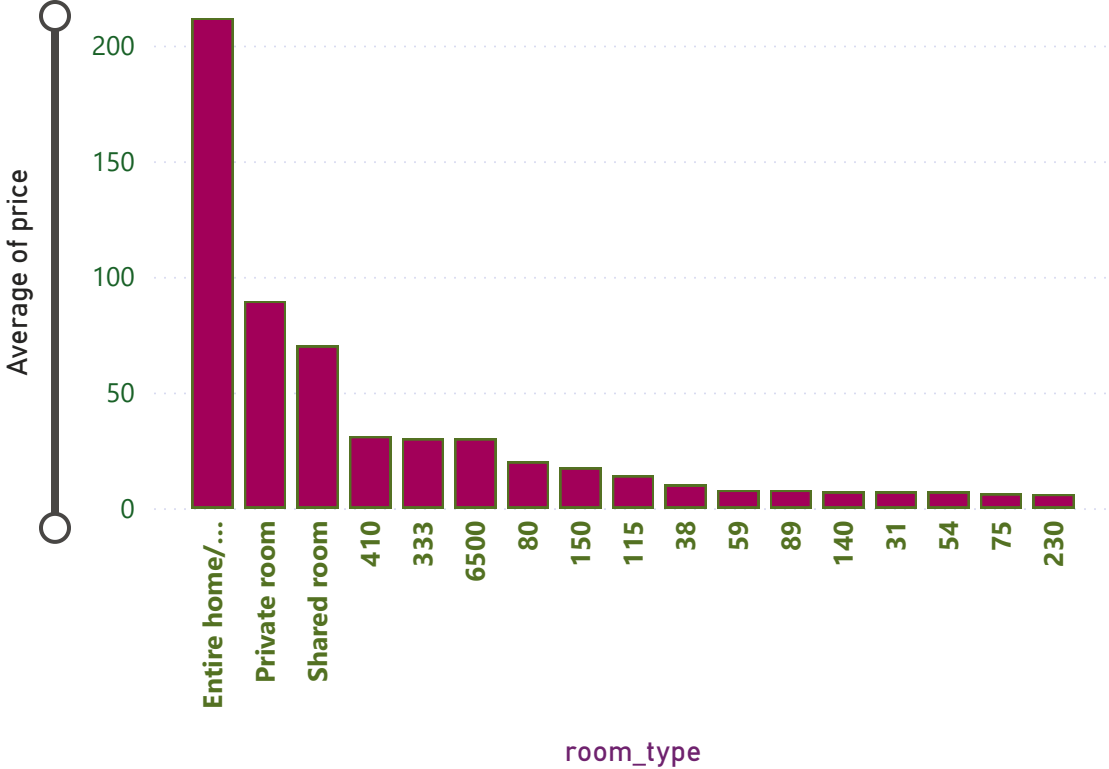


Sum of id by neighbourhood

neighbourhood



Average of price by room_type

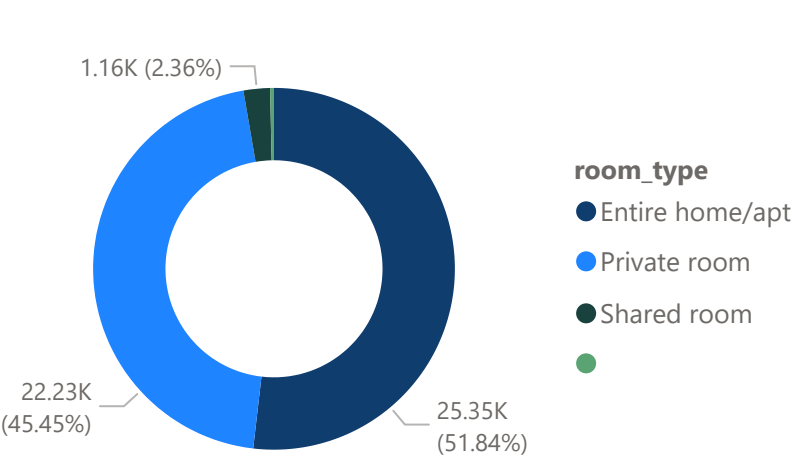


neighbourhood_group

- ☐
- ☐ 197400421
- ☐ Arverne
- ☐ Astoria
- ☐ Bath Beach
- ☐ Bedford-Stuyvesant
- ☐ Borough Park
- ☐ Briarwood
- ☐ Bronx



Count of id by room_type



Count of price
48.892K

Build visuals with your data

Select or drag fields from the **Data** pane onto the report canvas.

